JAXAtari Design Guide: Environment Implementation

TECHNISCHE UNIVERSITÄT DARMSTADT

November 12, 2025

This design guide will serve as an introduction to JAXAtari´s environment structure. This entails a description of the goals, general guidelines the environments should adhere to, specific functions they have to implement and the structure they have to adhere to.

1 Core Philosophy and Goals

This section outlines the high-level objectives for any JAXAtari environment.

- **Performance First**: The primary goal is to create highly performant and JAX JIT compatible Atari reimplementations designed for massive parallelization.
- **Behavioral Fidelity**: Environments should closely mimic the gameplay mechanics and behavior of the original Arcade Learning Environment (ALE) versions, though one-to-one visual or logical parity is not strictly necessary, for example visual effects that are not absolutely necessary to the gameplay or perfect replications of complex movement patterns are not expected.
- **Modifiability**: The code should be structured in a way that is clear and easy to modify, allowing for future research into environment variations.
- **JAX Native**: All logic must be implemented using JAX to leverage its core features like JIT compilation, automatic vectorization (`vmap`), and GPU/TPU execution. As we will explain in detail later on this includes the entire gameplay loop and rendering, but not setup functions for preprocessing.

2 Environment Anatomy: Core Components

This section breaks down the essential building blocks of a JAXAtari environment. Every environment is composed of a main class and several `NamedTuple` data structures.

- **The Main Environment Class**: Each game is a class that inherits from a base `JaxEnvironment` class, implementing the core gameplay logic.
- **`Constants` NamedTuple**: This structure holds all static, non-learnable parameters of the game, such as screen dimensions, player speed, or colors.
- **`State` NamedTuple**: This is the most critical component. It holds all dynamic variables that define the current state of the game (e.g., player position, score, ball velocity). The values inside the state are what changes inside the steps and it is always part of the input and the return of the step function.
- **`Observation` NamedTuple**: This structure holds the object-centric data exposed to the RL agent. Its specific content is game dependent and should contain everything the environment developer deems necessary knowledge to be able to play the game (position of player, position of enemies, etc).
- **`Info` NamedTuple**: This is used for carrying auxiliary diagnostic information that is not used for training but might otherwise be relevant, such as the current level.

3 The Environment Interface

All JAXAtari environments must implement the `JaxEnvironment` abstract class. This class defines a standard API for interaction, similar to other popular reinforcement learning frameworks. Adhering to this interface is essential for ensuring that environments are JIT-compatible and can be seamlessly used by agents and wrappers. Compatibility of environments can be tested locally using the provided tests in the tests/ folder. They can be executed for a specific game by using 'pytest –game [game_name]' where game_name is everything after 'jax_' in the games filename.
On this note this also means that all new environments need to be inside the src/jaxatari/games folder and adhere to the common naming scheme: 'jax_[game_name]'.

Additionally for environments to be compatible, every function defined in the `JaxEnvironment` base class must be implemented. These functions are detailed below.

3.1 `__init__`

Purpose The constructor is responsible for all one-time setup of the environment. This logic runs once on the CPU when the class is instantiated and is **not** JIT-compiled. Its primary role is to set up the game's static constants and instantiate the game-specific renderer. This is also a good place to pre-process data that can be used during execution to increase performance, for example pre-computing level architecture instead of doing it on the fly.

Parameters

- `consts`: An instance of the environment's specific `Constants NamedTuple`. If `None` is provided, the constructor should initialize a default version.

3.2 `reset`

Purpose This function resets the environment to its initial state, which is necessary at the beginning of every new episode. It must be JIT-compatible.

Parameters

- `key`: A `jrandom.PRNGKey` for environments that have stochastic starting conditions (though many Atari games are deterministic).

Returns A tuple of `(EnvObs, EnvState)` containing the initial observation for the agent and the complete initial state of the environment.

3.3 `step`

Purpose This is the main part of the environment. It takes a single action and advances the game logic by one frame. This function must be fully JIT-compatible and is where the core game logic resides. As described in Section 4, this function should ideally be implemented as an "orchestrator" that **only** calls internal, JIT-compatible helper functions (e.g., `_player_step`, `_ball_step`).

Parameters

- `state`: The complete `EnvState` object from the *previous* step.
- `action`: The action selected by the agent (e.g., an integer, for mapping see the JAXAtariAction class in environment.py).

Returns A tuple of `(EnvObs, EnvState, float, bool, EnvInfo)` containing:

- The new observation for the agent.
- The complete new `EnvState` object.
- The scalar reward obtained during this step.
- A boolean `done` flag, which is `True` if the new state is terminal.
- An `EnvInfo` object for auxiliary data.

3.4 `render`

Purpose This function generates a single RGB image (as a JAX array) representing the current game state. It is used for visualization and for agents that learn from pixels. This method should contain no game logic; it should only delegate the rendering task to the environment's dedicated `JAXGameRenderer` class.

Parameters

- `state`: The `EnvState` object to be rendered.

Returns A `jnp.ndarray` representing the RGB image.

3.5 `action_space`

Purpose A non-JIT helper function that defines the set of all valid actions an agent can take.

Returns A `Space` object (e.g., `spaces.Discrete`) that describes the action space.

3.6 `observation_space`

Purpose A non-JIT helper function that defines the structure, data types, and bounds of the object-centric `EnvObs`.

Returns A `Space` object (typically `spaces.Dict`) that describes the observation space.

3.7 `image_space`

Purpose A non-JIT helper function that defines the structure, data types, and bounds of the image returned by `render()`.

Returns A `Space` object (typically `spaces.Box`) that describes the image space.

3.8 `_get_observation`

Purpose An internal JIT-compatible helper function, usually called by `step`, that converts the full, internal `EnvState` into the public-facing `EnvObs`. This is used to filter out internal state variables that are not relevant to the agent.

Parameters

- `state`: The `EnvState` object of the current step

Returns The corresponding `EnvObs` object.

3.9 `obs_to_flat_array`

Purpose A JIT-compatible utility function that converts the structured, object-centric `EnvObs` (which is often a `NamedTuple` or `Dict`) into a single, flat 1D `jnp.ndarray`. This is required for agents that cannot process structured observations.

Parameters

- `obs`: The `EnvObs` object to flatten.

Returns A 1D `jnp.ndarray`.

3.10 `_get_info`

Purpose An internal JIT-compatible helper function, called by `step`, that extracts auxiliary information from the `state`. This data is not meant for the agent but is useful for logging or debugging (e.g., current lives, score, time).

Parameters

- `state`: The new `EnvState`.

Returns The `EnvInfo` object.

3.11 `_get_reward`

Purpose An internal JIT-compatible helper function, called by `step`, that calculates the scalar reward for the transition *from* `previous_state` *to* `state`.

Parameters

- `previous_state`: The `EnvState` from the prior step.
- `state`: The new `EnvState` after the action was taken.

Returns A `float` reward value.

3.12 `_get_done`

Purpose An internal JIT-compatible helper function, called by `step`, that determines if the new `state` is a terminal "game over" state.

Parameters

- `state`: The new `EnvState` to check.

Returns A `bool` which is `True` if the state is terminal, `False` otherwise.

4 JAX-Specific Implementation Guidelines

This section provides the crucial "rules" for writing code *inside* the environment methods to ensure it is JIT-compatible and performant. Also if you have no experience with JAX read the Getting Started subsections of the official JAX documentation first.

Immutability State variables cannot be modified in place (e.g., `state.player_y += 1` is forbidden). Instead, a new `State` object must be created with the updated value at every step.

JAX Control Flow

- **Conditionals**: Python's `if/else` statements must be replaced with `jax.lax.cond` for conditional logic within JIT-compiled functions.
- **Loops**: Python's `for` loops must be replaced with `jax.lax.fori_loop` for fixed-iteration loops.

Vectorization with `vmap` `jax.vmap` is the primary tool for parallelization. Instead of looping over a list of enemies, you should write a function to update a single enemy and then use `vmap` to apply it to all enemies simultaneously. Everything that can be parallel, should be parallel. Sequential execution is most often a major bottleneck.

Pure Functions Any function decorated with `@jit` should be "pure"—its output should depend only on its inputs, with no side effects. That also applies to all helper functions called inside such a function. It is not always necessary to decorate the helpers with `@jit` separately, although the difference in performance between the two options will most likely be minimal. **The step and reset functions should always be decorated with `@jit`!**

Decompose Logic into Helper Functions To support the goal of modifiability, the main `step` function should act as an orchestrator, not a monolith. Its primary role should be to call a series of smaller, self-contained helper functions, each responsible for a specific piece of game logic (e.g., `_player_step`, `_enemy_step`, `_ball_step`, `_timer_step`). This clean separation is crucial for the modification patterns described in Section 6, as it allows a researcher to easily override a single behavior (like `_child_step` in Kangaroo) by subclassing the environment, without needing to reimplement the entire `step` function.

Separating Static and Dynamic Logic (`__init__` vs. `step`)

- **Principle**: To maximize performance, any data or calculation that does not change from one frame to the next should be performed only **once** during the environment's initialization (`__init__`). The JIT-compiled `step` function should only contain logic that depends on the dynamic `state` and `action`.
- **Example (Level Data)**: For a game like Kangaroo, the positions and dimensions of all platforms on a given level are fixed. This data should be loaded or calculated in the `__init__` method. The `step` function can then read these static platform positions without having to recalculate them every frame.
- **Impact**: This separation is crucial. It reduces the computational load within the highly-optimized `step` function and allows the JIT compiler to treat static data as true constants, leading to better optimization.

5 Example Walkthrough: The Pong Environment

This final section ties everything together by walking through the implementation of `JaxPong` as a concrete example. Pong, Breakout, Kangaroo, Seaquest and Freeway are the best environments to look at for guidance as they are the most compliant to the guidelines.

Defining the Structures The environment first defines its core data structures using `NamedTuple`, including `PongConstants` for fixed values like paddle size and `PongState` for dynamic values like ball position and score. Also all relevant methods from the parent class are implemented according to their described functionality in section 3.

The `step` Function The main `step` function orchestrates the game logic. It very strongly follows the decomposition guideline by calling internal, JIT-compatible helper functions for all calculations and only handles passing the state between them. This extreme level of separation of logic is not always possible (or sensible) as always passing around the full state will be a performance issue for large games. For new environments try to decompose the logic into helper functions as far as possible (or reasonable for your specific environment) to make the later addition of modifications easier. Almost all of the current environments still have more or less decomposed structures which will be adjusted in the future.

A JIT-Compatible Helper The `_player_step` function demonstrates the JAX guidelines in practice. It uses `jax.lax.cond` to handle different player inputs (`up`, `down`) and game conditions (like touching a wall) in a way that can be efficiently compiled by JAX. This avoids standard Python `if/else` statements, ensuring the logic is performant and parallelizable.

6 Modifying Environments

A key design goal of JAXAtari is to be easily modifiable for future research. This allows for controlled experiments, such as simplifying a game, disabling certain mechanics, or modifying game elements.
All of our modifications are loaded through a single, unified pipeline. You pass your mod list directly to jaxatari.make():

```
env = jaxatari.make("pong", mods_config=["random_enemy"])
```

This command automatically builds the full two-stage modding pipeline:
Stage 1 (JaxAtariModController): Manages all internal mods (like changing game logic, assets, or constants).
Stage 2 (JaxAtariModWrapper): Manages all post-step mods that run after the step is complete.
This pipeline works via a plugin system. To create a mod, you create a plugin class.

6.1 Step 1: Choose Your Mod Type

You must choose one of two base classes for your plugin.

Type 1: JaxAtariInternalModPlugin (For Internal Logic) This is the most powerful type, used to change the game's core mechanics. It allows you to:

- Patch internal functions (like _enemy_step).
- Override member attributes (like frameskip).
- Override constants (like PLAYER_COLOR).
- Override assets (like player.npy). [1]

The controller finds plugin methods by name. If your plugin defines a function named **_enemy_step**, it patches the environment's internal **_enemy_step** function.

```
class LazyEnemyMod(JaxAtariInternalModPlugin):
    conflicts_with = ["random_enemy"]
    '''
    Examples of how to use the constant and attribute replacement
    '''
    constants_overrides = {
        "PLAYER_ACCELERATION": jnp.array([6])
    }
    attribute_overrides = {
        "obs_size": 0
    }

    @partial(jax.jit, static_argnums=(0,))
    def _enemy_step(self, state: PongState) -> PongState:
        """
        Replaces the base _enemy_step logic.
        Access the base environment itself via self._env (set by JaxAtariModController).
        """
        should_move = (state.step_counter % 8 != 0) & (state.ball_vel_x < 0)
        direction = jnp.sign(state.ball_y - state.enemy_y)
       new_y = state.enemy_y + (direction * self._env.consts.ENEMY_STEP_SIZE).astype(jnp.int32)

      final_y = jax.lax.cond(should_move, lambda _: new_y, lambda _: state.enemy_y, operand=None)
        return state._replace(enemy_y=final_y.astype(jnp.int32))
```

Type 2: JaxAtariPostStepModPlugin (For Post-Step Logic) This is used for simple, non-invasive changes that should happen after the step logic is finished (e.g., changing rewards or freezing a state value). To do this you just need to implement a run() function inside the plugin that is always run directly after the environments step() function. If you want to add logic after the base environment reset() function you can also implement an after_reset() function inside the plugin which works just like the run() function.

```
class AlwaysZeroScoreMod(JaxAtariPostStepModPlugin):
    @partial(jax.jit, static_argnums=(0,))
    def run(self, prev_state, new_state):
        """
        This function is called by the wrapper *after*
        the main step is complete.
        Access the environment via self._env (set by JaxAtariModWrapper).
        Also provides the prev_state in case persistence is required in any form.
        """
        return new_state._replace(
```

[1] To ensure asset_overrides can function, the asset manifest must be defined within the environment's Constants NamedTuple. This allows the modding controller to patch the asset list before the renderer is initialized and loads any sprites. The standard implementation pattern, where the renderer's __init__ accesses this manifest via self.consts.ASSET_CONFIG, is detailed with examples in the JAXGameRenderer Design Guide.

```
            player_score=jnp.array(0, dtype=jnp.int32),
            enemy_score=jnp.array(0, dtype=jnp.int32)
        )
```

6.2 Step 2: Adapting the base environment (If Needed)

Our modding framework is a patcher. It cannot add new information that the base environment doesn't already have. If your mod needs new state (like a PRNGKey) or a new hook (like a _render_hook_post_ui), you must perform a one-time modification to the base environment files.

Example: Adding a PRNGKey to jax_pong.py

- Modify the State: Add key: chex.PRNGKey to the PongState(NamedTuple) definition.
- Modify reset(): Add key management to reset() to initialize the state_key.
- Modify step(): Add key management to step() to split the key.

Your InternalModPlugin (like RandomEnemyMod) can now use the key inside the normal game state.

6.3 Step 3: Register and Load the Mod

Once your plugin is written, you just add it to your game's REGISTRY.

```
class PongEnvMod(JaxAtariModController):

# This REGISTRY is the single source of truth
REGISTRY = { "lazy_enemy": LazyEnemyMod, "zero_score": AlwaysZeroScoreMod, }

def __init__(self, env, mods_config: list = [], ...):
    super().__init__(
        env=env,
        mods_config=mods_config,
        registry=self.REGISTRY,
        ...
    )
```

The jaxatari.make() function will now automatically find and apply your mod when a user requests it.